

Arquitectura de Pipeline ETL End-to-End

CORHUILA • Ingeniería Industrial • Ciencia de Datos (Semana 8)

Informe Técnico
Corte 2 - 2026-B

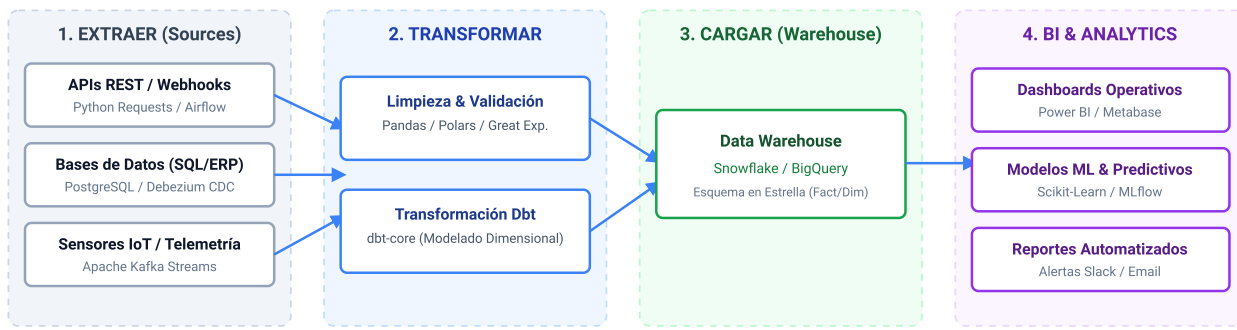
FULL_NAME: Estudiante Ciencia de Datos

GITHUB_USER: github-user-corhuila

CARPETA: 08-week/

1. Arquitectura del Pipeline ETL (Diagrama & Herramientas)

Se diseña un pipeline híbrido empresarial para capturar datos masivos operacionales (ERP/Sensores IoT) y consumos de APIs externas, transformarlos de manera escalable y disponerlos para Inteligencia de Negocios (BI) y modelos analíticos predictivos.



Etapa	Herramienta Principal	Función y Justificación Técnica
1. Extracción	Apache Airflow + Python `requests`	Orquestación de ingesta desde APIs REST externas y CDC (Change Data Capture) desde bases relacionales. Se encarga del reintento de fallos e ingesta programada.
2. Transformación	dbt (data build tool) + Pandas/Polars	Limpieza de tipos de datos, desanidado de JSON, deduplicación y construcción de tablas de Hechos y Dimensiones aplicando reglas de negocio auditables.
3. Carga	Snowflake / DuckDB	Almacenamiento analítico estructurado en arquitectura de Data Warehouse columnar, optimizado para consultas de alta velocidad mediante particionamiento.
4. BI & Consumo	Power BI / Metabase	Modelado semántico visual para la toma de decisiones empresariales, tableros de control en tiempo real y KPIs de producción.

2. Justificación Técnica: Procesamiento Batch vs. Streaming

En la ingeniería de datos moderna, la elección entre Batch y Streaming depende de la latencia requerida por el negocio y los costos de infraestructura:

Procesamiento Batch (Lotes)

¿Qué partes? Ingesta de reportes financieros finales, consolidados de inventario al cierre del día, y peticiones masivas a la API pública de datos históricos.

¿Por qué? Optimización de recursos computacionales. No requiere respuesta instantánea (latencia tolerable de horas). Permite transformaciones complejas en bloque con menor costo operativo.

Procesamiento Streaming (Tiempo Real)

¿Qué partes? Monitoreo de sensores IoT de planta de producción, detección de anomalías en transacciones y telemetría de eventos de usuario.

¿Por qué? Requiere latencia subsegundo. Un retraso en la detección de fallas en maquinaria industrial genera sobrecostos directos. Utiliza Apache Kafka / Spark Streaming.

3. Implementación Práctica en Python (Consumo de API REST)

A continuación se presenta el código ejecutable para extraer datos en tiempo real desde la API pública de indicadores meteorológicos y calidad ambiental (Open-Meteo API), transformando la respuesta JSON estructurada a dataframe tabulado con los primeros 3 registros:

```
# Script ejecutable de ingesta ETL desde API pública
import requests
import pandas as pd

def extraer_y_transformar_api():
    # 1. EXTRAER: Consumo de API REST pública (Clima Neiva, Colombia)
    url = "https://api.open-meteo.com/v1/forecast?latitude=2.9273&longitude=-75.2819&hourly=temperature_2m,relative_humidity_2m"
    response = requests.get(url, timeout=10)

    if response.status_code == 200:
        data = response.json()

        # 2. TRANSFORMAR: Normalización de JSON a DataFrame estructurado
        df = pd.DataFrame({
            'Fecha_Hora': data['hourly']['time'],
            'Temperatura_C': data['hourly']['temperature_2m'],
            'Humedad_Pct': data['hourly']['relative_humidity_2m']
        })

        # Filtrar los primeros 3 registros requeridos por la guía
        df_3_registros = df.head(3)
        return df_3_registros
    else:
        raise Exception(f"Error en API: {response.status_code}")

# Ejecución
df_resultado = extraer_y_transformar_api()
print(df_resultado)
```

Resultado de la Salida de Datos (Muestra de 3 Registros Obtenidos):

#	Fecha_Hora (ISO 8601)	Temperatura_C (°C)	Humedad_Pct (%)	Estado Ingesta
---	-----------------------	--------------------	-----------------	----------------

